

EE2028: Microcontroller Programming & Interfacing

Academic Year 2025/2026 Semester 1



NUS
National University
of Singapore

Assignment 1 Project Report

Matthew Goh Jin Han	A0308358X
Jeremy Lim Zhen Xing	A0308348Y

Project Logic Flow:

- **Prologue / Setup**

- Save callee-saved registers (r4–r12, lr) to follow ARM ABI.
- Copy function arguments into working registers: N, coefficient pointers, and input x_n.
- Load base addresses of history arrays x_store and y_store.

- **Base Term Calculation**

- Compute the initial accumulator:

$$y_n = \frac{x_n * b[0]}{a[0]}$$

- This forms the starting point of the current output.

- **Summation Loop (j = 0 .. N-1)**

- For each tap:
 - Load b[j+1] and a[j+1].
 - Load delayed input x_store[j] and delayed output y_store[j].
 - Compute contribution:
$$\frac{b[j+1]*x_store[j]-a[j+1]*y_store[j]}{a[0]}$$
 - Add contribution to accumulator y_n.

- **Shift Histories**

- Move old values backwards (N-1 down to 1).
- Prevents overwriting and keeps the correct delay line.

- **Insert Current Values**

- Store current input x_n into x_store[0].
- Store unscaled output y_n into y_store[0].

- **Scaling and Return**

- Divide y_n by 100 (to correct coefficient scaling).
- Restore saved registers and return the result in r0

Question 1:

Knowing the starting address of a 2-d array `Arr[][]`, how to calculate the memory address of element `Arr[A][B]` with index A and B starting from 0? Use an equation to explain your answer. (1 mark)

Memory Address of `Arr[A][B]` = Starting Address + $(A \times \text{Number Of Columns} + B) \times \text{Size Of Element}$

Question 2:

Consider the following C code snippet and the corresponding assembly routine:

C Code:

```

27 #include <stdio.h>
28
29 extern int foo(int arg1, int arg2);
30
31 int main() {
32     int a = 5, b = 10;
33     int result;
34
35     result = foo(a, b);
36     printf("Result: %d\n", result);
37
38     return 0;
39 }

```

Assembly Routine:

```

30 foo:
31     PUSH {R14}
32
33     BL SUBROUTINE
34
35     POP {R14}
36     BX LR
37
38 SUBROUTINE:
39
40     BX LR

```

Read the above code. Assume each line of the C code corresponds to a single line of assembly instruction. After executing the highlighted line (Ln 36, BX LR), which instruction is the Link Register (LR) pointing to at this moment? (1 mark)

At line 36 (BX LR) inside foo, LR holds the return address to the caller, the instruction after `result = foo(a, b);` in main.

So LR points to the instruction that executes the `printf("Result: %d\n", result);` line.

Question 3:

Describe what you observe in (i) and (ii) and explain why there is a difference. (2 marks)

Without the push and pop, the assembly program would not be able to return to the C code.

Question 4:

What can you do if you have used up all the general-purpose registers and you need to store some more values during processing? (1 marks)

When all general-purpose registers are used, we can store extra values on the stack (by pushing them at function entry) and later recover them (by popping at function exit).

Question 5:

Consider the following C code snippet:

```
27 #include <stdio.h>
28
29 int main() {
30     int a = 5, b = 10;
31     float result;
32
33     if (a < b) {
34         result = a / 2.0;
35     } else {
36         result = b / 2.0;
37     }
38
39     printf("Result: %.1f\n", result);
40     return 0;
41 }
```

Explain what the code is intended to do, focusing on the if-else structure. What is the output of the code? (1 mark)

It will first check if variable a is less than b

if true, a will be divided by 2.0, and the output (2.5) will be placed into the result. However, if it's not true, b will be divided by 2.0, and the output (5.0) will be placed into the result.

Afterwards, the contents of the result would be printed as an output, and the program would be terminated (due to returning 0)

Discuss how the data type of the result affects the output when changing the values of a and b. What would happen if the result were declared as an integer? You might need to make appropriate changes to the printf() function call. (1 mark)

Declaring the result as a float allows decimal precision. If it were declared as an int, the fractional part would be truncated. For example, if the result is 2.5, it would only show 2. The printf call would also need to change from %f to %d to show the correct integer value.

Improvements Made:

Register Reuse and Optimisation

Frequently accessed variables such as N, base pointers to a and b, and the current input x_n are stored in registers (r4–r7) rather than reloading them from memory repeatedly.

Temporary registers (r1–r3, r11–r12) are reused across different parts of the program (loop counters, offsets, intermediate products), reducing the number of instructions required and lowering register pressure.

Constant Value Hoisting

The divisor a[0] is loaded once into r10 and reused for all divisions. This avoids redundant memory accesses inside the summation loop.

Similarly, base addresses of x_store and y_store are preloaded into r8 and r9 for repeated use.

Efficient Loop Structure

The summation loop uses cmp and bge for a clear termination condition ($j < N$), avoiding unnecessary iterations.

The shift loop runs backwards (from $N-1$ down to 1), which ensures correct updating of history buffers without needing extra temporary storage.

Static Memory Allocation for Delay Lines

History arrays (x_store, y_store) are statically allocated using .lcomm, ensuring persistence across multiple function calls without repeated initialisation.

Appendix:

All members in this group have contributed to this assignment.

Matthew Goh Jin Han A3038358X

Jeremy Lim Zhen Xing A0308348Y